

Git Mental Model

1. What Git actually stores

Git primarily stores a **graph of commits**.

A commit contains:

- SHA (unique ID)
- Parent commit(s)
- Snapshot of all files
- Metadata (author, timestamp, message)

Example:

```
A -> B -> C
```



means:

```
C.parent = B  
B.parent = A
```



Commits are **immutable**. Once created, they never change.

2. What is a branch?

A branch is just a **named pointer to a commit**.

Example:

```
A -> B -> C  
      ^  
    main
```



means:

```
main = C
```



Nothing more.

A branch does not contain commits.

A branch does not contain files.

A branch is just a reference.

3. Creating a branch

Starting from:

```
A -> B -> C
      ^
      |
    main
```

Create branch:

```
</> Bash
git branch feature
```

Result:

```
A -> B -> C
      ^ ^
      | |
    main feature
```

Both pointers reference the same commit.

No commits are copied.

4. Does Git remember branch origins?

No.

Git does NOT store:

```
feature was created from main
```

It only stores:

```
main = C
feature = C
```

Later, Git infers relationships from commit ancestry.

Merge

Example

Before merge:

```
main  
  
A -> B -> C  
  
feature  
  
A -> B -> D -> E
```



Common ancestor:

```
B
```



After merge:

```
      D -> E  
     /    \  
A -> B -> C -> M
```



Merge commit:

```
M.parent1 = C  
M.parent2 = E
```



A merge commit has **two parents**.

Why merge exists

Merge preserves the true history.

Git can still see:

```
main evolved to C  
feature evolved to E  
they were combined at M
```



No history is rewritten.

What commits become reachable from main?

After merge:

```
      D -> E
     /     \
    A -> B -> C -> M
```



Commits reachable from main:

```
A B C D E M
```



because Git can walk both parent chains.

This is why Git knows D and E were merged.

Rebase

Example

Before:

```
main

A -> B -> C -> X -> Y

feature

A -> B -> C -> D -> E
```



Rebase feature onto main:

```
</> Bash

git rebase main
```



Result:

```
A -> B -> C -> X -> Y -> D' -> E'
```



Git creates new commits:

```
D'
E'
```



with new SHAs.

Important

Git does NOT move commits.

Original commits still exist:

```
D  
E
```



Git creates:

```
D'  
E'
```



and moves the branch pointer.

Why SHAs change

Commit SHA depends on:

- snapshot
- parent(s)
- metadata

Since:

```
D.parent = C
```



but

```
D'.parent = Y
```



SHA changes.

Purpose of rebase

Rebase rewrites history to make it appear as if work started from a different commit.

It produces a clean linear history.

Squash

Example

Feature branch:

```
A -> B -> C -> D -> E
```



Suppose:

```
C = add API  
D = fix bug  
E = review changes
```



Squash creates:

```
A -> B -> S
```



where S contains the final snapshot of E.

Important

Git does NOT preserve:

```
C  
D  
E
```



on the target branch.

Instead Git creates a completely new commit:

```
S
```



with a new SHA.

Key insight

Even if:

```
files(E) == files(S)
```



Git still sees:

E != S



because SHAs and ancestry differ.

Pull Request Internals

A PR consists of two separate calculations:

1. Commit list
2. File diff

These are not calculated the same way.

How PR commits are determined

Suppose:

main

A -> B -> C

feature

A -> B -> D -> E



GitHub effectively asks:

</> Bash

```
git log main..feature
```



Meaning:

Show commits reachable from feature but not reachable from main.

Result:

D

E



This is ancestry-based.

How PR file diffs are determined

GitHub finds the merge base.

Example:

```
main  
  
A -> B -> C  
  
feature  
  
A -> B -> D -> E
```



Merge base:

```
B
```



GitHub effectively compares:

```
</> Bash  
  
git diff B E
```



or conceptually:

```
merge-base(base, feature)  
  ↓  
feature HEAD
```



Result:

```
Changes introduced by feature branch
```



Squash Merge Confusion

Suppose:

```
main  
  
A -> B -> S  
  
feature
```




```
A -> B -> C -> D -> E
```

where:

```
S contains the same code as E
```

Commit comparison

GitHub asks:

```
</> Bash
```

```
git log main..feature
```

Result:

```
C
D
E
```

because these commits are not reachable from main.

File comparison

Merge base is still:

```
B
```

GitHub compares:

```
B -> E
```

not:

```
S -> E
```

Therefore GitHub may still show:

```
Commits:
C D E
```

```
Files:
many changes
```

even though the code already exists in main.

Git does not know:

```
S == C+D+E
```



It only sees commit ancestry.

Most Important Takeaway

Git's primary model is:

```
Commit graph + branch pointers
```



not:

```
List of file changes
```



Git determines whether work is merged primarily through **commit ancestry (reachability)**, not by checking whether file contents happen to be identical.

This single idea explains most confusing Git behaviors involving merges, rebases, squash merges, and pull requests.